

Adaptive Computation Reuse for Energy-Efficient Training of Deep Neural Networks

JASON SERVAIS and EHSAN ATOOFIAN, Lakehead University

In recent years, Deep Neural Networks (DNNs) have been deployed into a diverse set of applications from voice recognition to scene generation mostly due to their high-accuracy. DNNs are known to be computationally intensive applications, requiring a significant power budget. There have been a large number of investigations into energy-efficiency of DNNs. However, most of them primarily focused on inference while training of DNNs has received little attention.

This work proposes an adaptive technique to identify and avoid redundant computations during the training of DNNs. Elements of activations exhibit a high degree of similarity, causing inputs and outputs of layers of neural networks to perform redundant computations. Based on this observation, we propose Adaptive Computation Reuse for Tensor Cores (ACRTC) where results of previous arithmetic operations are used to avoid redundant computations. ACRTC is an architectural technique, which enables accelerators to take advantage of similarity in input operands and speedup the training process while also increasing energy-efficiency. ACRTC dynamically adjusts the strength of computation reuse based on the tolerance of precision relaxation in different training phases. Over a wide range of neural network topologies, ACRTC accelerates training by 33% and saves energy by 32% with negligible impact on accuracy.

CCS Concepts: • **Software and its engineering** → **Message oriented middleware**; • **Computer systems organization** → **Neural networks**;

Additional Key Words and Phrases: Computation reuse, training, DNNs, accelerator, energy, performance

ACM Reference format:

Jason Servais and Ehsan Atoofian. 2021. Adaptive Computation Reuse for Energy-Efficient Training of Deep Neural Networks. *ACM Trans. Embed. Comput. Syst.* 20, 6, Article 114 (October 2021), 24 pages.
<https://doi.org/10.1145/3487025>

1 INTRODUCTION

We are witnessing the success of **Deep Neural Networks (DNNs)** in a wide range of applications with a major impact on the world economic and social activity. DNN-based systems are ubiquitous and so it is necessary to offer energy-efficient DNNs for current and future computing devices. While there have been numerous research works on the energy-efficiency of inference in DNNs, only a handful of techniques were proposed for training of DNNs. A decade ago, a neural-network could be trained using a commodity computer within a few hours. However, training a DNN now

Authors' address: J. Servais and E. Atoofian, Lakehead University, 955 Oliver Rd., Thunder Bay P7B 5E1, Canada; emails: {jmservai, atoofian}@lakeheadu.ca.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2021 Association for Computing Machinery.

1539-9087/2021/10-ART114 \$15.00

<https://doi.org/10.1145/3487025>

may take several weeks or even months and became an exascale class problem [11]. Training of contemporary DNNs requires GPUs or custom-built accelerators such as TPU [13] or Cerebras [14] so that they can be trained within a reasonable amount of time. In addition to the cost of purchasing of these devices or getting access to them, the other challenge is the environmental impact of training. Training of a DNN may create 39 lbs of CO₂ [12]. Furthermore, some edge devices require training of a **neural network (NN)** through refining of an existing model based on some constraints defined by a user. While design requirements of edge devices are different from those used in data centers or servers, all of them need to accelerate training and improve energy-efficiency under certain design constraints.

There have been a few efforts to reduce the execution time of training. Lossy and lossless compression techniques were used to reduce the memory footprint of training [5, 6] and enhance communication delay between memory and computing cores. Distributed training exploits several computing nodes and uses each node to run a subset of the entire training [7]. Utilizing sparsity in data [8, 18] and weight [2, 9] is another technique for the acceleration of training. Despite all these efforts, training remains an exascale problem and further optimizations are needed.

The bulk of delay as well as energy consumption during training is due to computations needed to perform dot products of rows and columns in matrices. Whether it is during the forward or backward phase, two matrices, i.e., input and weight, are multiplied. Here, the input refers to the input image for the first layer and activation map for the hidden layers. As an example, a convolutional layer is composed of several filters (or weights). The output is computed by sliding each filter across the input and taking a filter sized dot product at each position. When evaluated in hardware, the computation requires reading the input and filters from a memory and performing **multiply-accumulate (MAC)** on input-weight pairs. This can be modeled by a matrix multiplication. The input image is unfolded into a large matrix (A) and then is multiplied by the weight matrix (W). Prior works studied redundancy of MACs during inference [35, 38]. However, exploiting computation reuse during training is more challenging than inference. First, just because similar values appear during inference does not imply that they also occur during training. Training starts with random initialization of network parameters and continues with refining the parameters until the network converges. During the training, some values in tensors such as the activation map become equal. However, it is not known whether they stay equal until the end of the training. Second, position of the equal values in tensors during the inference is known in advance. Hence, for inference, we can pre-schedule the neuron computations to exploit the reusability of tensor values. This is not the case for the training phase as network parameters change during the training and it is not feasible to determine the pattern of computations in advance. Third, during the inference, only two tensors are used: input and filter. The two tensors are used only once and as a result, they can be laid out in the memory so that the order of values match the pattern of computations. However, during the training, a third tensor is involved: gradient. Also, each tensor is used twice in one pass of the training and the way each tensor is used each time is different. For example, for the forward pass, a different set of activations contribute to an output than the backward pass. As a result, it is challenging to arrange values in memory so that both forward and backward passes can benefit.

When redundancy exists, it provides an opportunity to improve energy efficiency for training of DNNs. We observe that during training, when two matrices are multiplied ($A \times B$), many rows (columns) in the first (second) matrix are similar. This opens up an opportunity to exploit computation reuse and reduce the number of MACs. As an example, the rows with the same color in Figure 1(a) are similar. So, instead of computing dot products of all rows in A and columns in W , we only need to compute dot products of half of those rows and columns. This cuts in half the MACs in hardware and increases energy-efficiency of training. We find that for many networks,

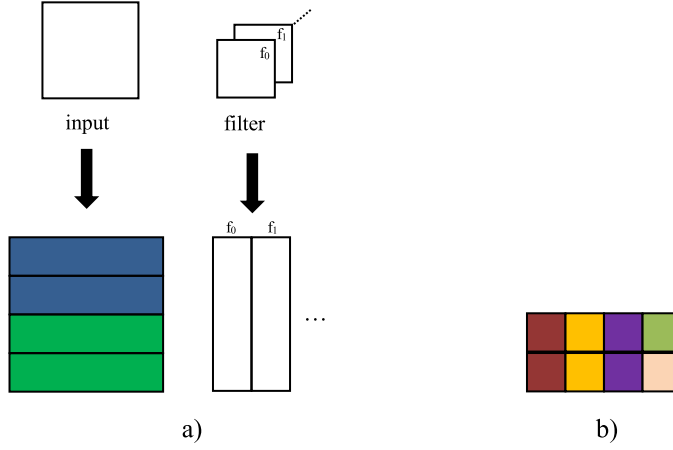


Fig. 1. (a) Convolution between an input and a filter. (b) An input with two rows. The first three elements of the two rows are the same.

similarity occurs in the activation during the forward and backward passes (details will appear in Section 3).

Despite the high degree of similarity, floating-point numbers across rows/columns are not exactly the same. Whether full-blown floating-point operands (32-bit) or low-precision operands (i.e., 16-bit) are used during the training, it is unlikely that a significant number of rows (columns) be equal. However, neural networks are resilient to errors [16] and slightly changing operands in matrix multiplications has negligible impact on accuracy. We exploit this property to boost the potential of computation reuse in training of DNNs. In particular, we exploit linear quantization [4] to convert similar values to equal values. In this work, we show that by applying linear quantization to a set of DNNs selected from a wide range of applications, a significant fraction of MACs has the same quantized value while quantization has a marginal impact on accuracy.

Based on the high degree of similarity in rows (columns) of matrices, we propose a mechanism for computing dot products of matrices by reusing the buffered results. We explain the mechanism through an example. Let us assume that filters of a convolutional layer are represented by a 4×4 matrix and the input image has two rows. The first three elements of the two rows in the input image are the same ($x_{00} = x_{10}$, $x_{01} = x_{11}$, and $x_{02} = x_{12}$, the first index is the row number and the second index is the column number) and only the last element is different ($x_{03} \neq x_{13}$) (Figure 1(b)). Elements with the same color in Figure 1(b) indicate that they are the same. Hence, we can use computation reuse and avoid most of the MACs for the second row. One approach for exploiting similarity in the input image is that we save source operands of dot products of the first row of the image and the filter (weight). Since the dimension of the filter is 4×4 , we should allocate space for 16 MACs. Then, for the second row, we can skip MACs corresponding to the first three elements (x_{10} , x_{11} , and x_{12}) as they are the same as the first row. This reduces the number of MACs for the second row from 16 to 4. While this technique reduces the number of multiplications, it requires extra buffers to hold the outcome of individual MACs from the first row.

The alternative approach would be saving the outcome of dot products in buffers and subtract the inputs that are different and add the new ones (adjusted by their respective weights). We explain this through an example. Let us assume that the outcome of $A \times W$ is represented by Z . We can compute z_{10} based on z_{00} : $z_{10} = z_{00} + (x_{10} - x_{00}) \times w_{00} + (x_{11} - x_{01}) \times w_{10} + (x_{12} - x_{02}) \times w_{20} + (x_{13} - x_{03}) \times w_{30}$. Since the first three elements of the two rows in A are the same, computation of z_{10}

is simplified to: $z_{10} = z_{00} + (x_{13} - x_{03}) \times w_{30}$. Note that for this example, z_{10} requires fetching only one weight from internal memory instead of four. In addition, only one/multiplication/addition is needed instead of four/three. While this method reduces the number of multiplications/additions for similar rows, it requires extra subtractions. However, the subtractions are computed once for the second row and are used for all columns of the filter matrix (W). Hence, for those rows of the input image that exhibit value similarity, it is more efficient to compute dot products using the second technique rather than saving outcomes of individual MACs.

In this article, we propose a low-overhead hardware scheme that exploits computation reuse and improves energy-efficiency in DNNs. We extend the architecture of a state-of-the-art accelerator and add extra buffers to save outputs of dot products and reuse them in the future. Our proposed scheme quantizes matrices and then it computes the outputs by using only the inputs that have been changed with respect to the saved values. This prevents access to internal buffers as well as computations for matrix values that remain unmodified. We also present hardware components as well as a low-cost data interconnect to shuffle MACs and replace redundant MACs with useful computations.

The extra hardware needed for the implementation of our technique is modest as components available in the baseline accelerator are employed to support most of the operations needed for computation reuse. In addition to changes to the control unit needed for skipping redundant MACs, our technique requires extra storage to save outputs of dot products and additional multiplexers to shuffle MACs. The extra components represent a small increase in accelerator resources. Our proposed scheme does not require any modification in the application layer and works with existing network models without any specific annotation.

In summary, the main contributions of this article are as follows:

- We analyze the degree of similarity in forward and backward passes of DNNs selected from a wide range of applications. We show that on average, 70% of MACs in training have the same quantized value.
- We propose a low-overhead hardware to exploit similarities occurring during training which appears predominantly in the activations. Our proposed scheme detects similar values and shuffles MACs in time and space so that overall computation finishes earlier. We do not make any assumption on how similar values are distributed so that the underlying hardware can efficiently handle the dynamic similarity that arises during the training.
- Our proposed scheme is compatible with any accelerator that performs multiple MAC operations in parallel and accumulates all of them into a single output. Our hardware additions are moderate and can be smoothly merged with existing accelerators.
- We implement a reuse-based training on top of an accelerator based on **Tensor Core (TC)** processing units [17]. We show that our technique improves performance by 33% and energy by 32% on average over a set of deep learning models covering a wide range of applications.

The rest of the article is organized as follows. Section 2 briefly describes training of DNNs. Section 3 presents similarity of matrices used in forward and backward passes. It also discusses the impact of quantization on accuracy of reused-enabled DNNs. Section 4 proposes a detailed architecture to improve efficiency through computation reuse. We present the architecture of TC and additional components needed to implement computation reuse. Section 5 presents the evaluation methodology and Section 6 discusses the experimental results. Section 7 reviews related work and finally, Section 8 concludes the article.

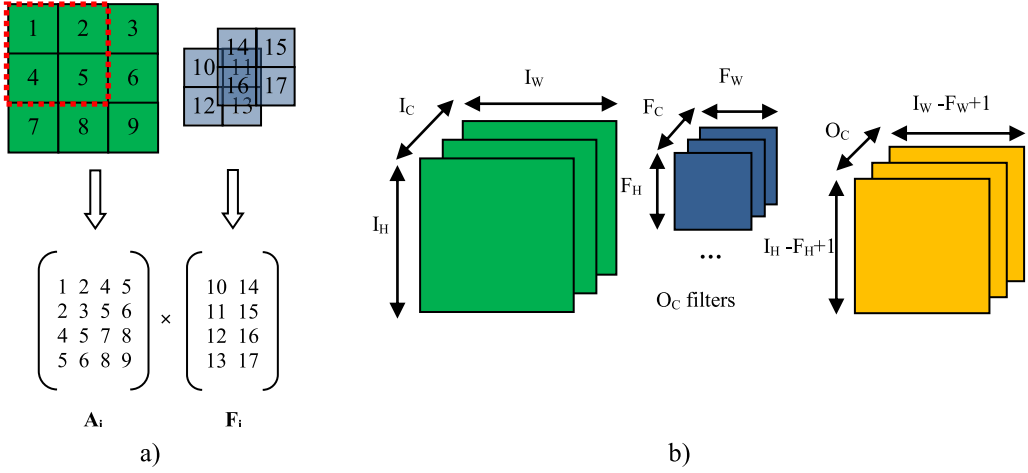


Fig. 2. (a) Convolution between an image and weight filters. (b) CNN parameters per convolutional layer.

2 BACKGROUND

In this section, we explain training only for convolutional layers as data flow in a convolutional layer is complicated due to the sliding window associated with multiple filters. However, our technique is applied to both convolutional and fully-connected layers as the two layers are responsible for the majority of delay and energy in DNNs.

To compute convolution of an input image and weight filters, the input image is unfolded into a large matrix and then is multiplied by a matrix corresponding to the filters. Figure 2(a) illustrates how the input image and filters are mapped to matrices. We assume a 3×3 input image and two 2×2 filters. Each row of the first matrix corresponds to the elements within a window from the input image. For example, the first window (represented by a red dashed line) includes four numbers: 1–5. These four numbers form the first row of the first matrix. Each column of the second matrix corresponds to elements of a filter. Since the dimension of each filter is 2×2 , there are four elements per column in the second matrix.

Training of a layer contains two steps: forward propagation and backward propagation. During the forward pass, the output of layer i is computed based on the formula below

$$A_{i+1} = A_i \times F_i, \quad (1)$$

where A_i is unfolded input activation, F_i is the weight matrix, all for layer i . The output of layer i is fed to the input of the next layer ($i + 1$). When performing the forward pass, the convolutional layer takes two inputs: an input activation with size $I_W \times I_H \times I_C$ and O_C 3D filters, each with size of $F_W \times F_H \times F_C$. Here, I_W , I_H , and I_C denote width, height, and the number of channels in the input activation. Similarly, F_W , F_H , and F_C represent width, height, and the number of channels in a filter. Convolutions between the inputs and filters form a 3D $(I_W - F_W + 1) \times (I_H - F_H + 1) \times O_C$ output. The number of channels in the output (O_C) is the same as the number of filters. These parameters are visualized in Figure 2(b).

The input is unfolded into a large input activation matrix A with a dimension of $N \times K$. The number of rows in A (N) is equal to $(I_W - F_W + 1) \times (I_H - F_H + 1)$ and the number of columns in A (K) is equal to $(F_W \times F_H \times I_C)$. The filters of a convolutional layer are represented by matrix F with a dimension of $K \times M$ where M is the number of filters (O_C). The output has a dimension of

Table 1. Notations used in this Article

Notation	Meaning
I_W	Width of input activation
I_H	Height of input activation
I_C	# of channel of input activation
F_W	The filter width
F_H	The filter height
F_C	# of channel of filter
O_C	# of channel of output
M	# of filters
N	# of rows in the input activation matrix
K	Size of a filter

$N \times M$ and is computed based on Equation (1). Hence, during the forward pass, the complexity of matrix multiplication is: $O(M \times N \times K)$. Table 1 shows the list of symbols used in this article.

During the backward pass, there are two key computations to perform: gradient of inputs and gradient of weight. The formulas for the backward pass are as follows:

$$G_{i-1} = G_i \times (F_i)^T, \quad (2)$$

$$G_{Fi} = (A_i)^T \times G_i \quad (3)$$

where G_{i-1} is the gradient of input activation, G_i is the gradient of output activation, and G_{Fi} is the gradient of weight. Similar to the forward pass, the key component of the backward pass is matrix multiplication. Since the dimension of G_i is $N \times M$, the complexity of backward propagation is $O(2 \times M \times N \times K)$.

Training of DNNs is based on the gradient descent algorithm where during the forward pass, Equation (1) is applied in sequence from the first layer all the way to the last layer. For each layer, the convolution of the input activation and filters is computed to generate output activation which is fed as input to the next layer. The output of the very last layer is used to find the prediction error (gradients) relative to the correct outputs. Then, during the backward pass, the gradients are backpropagated through network layers to update network parameters. During the backward propagation, the layers are invoked in reverse order from the last to the first layer. Each layer convolves its output gradient with filters to generate input gradient to be fed to the preceding layer. The layer also convolves its input activation with output gradient to generate weight gradient. The average of weight gradients across a batch is used to update weights for a pass based on Equation (4).

$$W_{i+1} = W_i - \alpha \times \text{average}_{\text{weight_gradients}}, \quad (4)$$

where W_{i+1} is the new weight, W_i is the old weight, and α is the learning rate.

3 VALUE SIMILARITY

In this section, we focus on similarity that exists across rows and columns of matrices involved in the training of DNNs and introduce a technique that exploits the value similarity to reduce computations and memory accesses. Table 2 shows the set of DNNs used for the evaluation of our proposed technique.

Table 2. The Set of Benchmarks Employed for Evaluation of ACRTC

Dataset	Network Topology	Accuracy
HAR	FC1: 512, FC2:512, FC3:12	93.6%
ISOLET	FC1: 512, FC2:512, FC3:26	95.6%
MNIST	CONV1: $20 \times 5 \times 5$, CONV2: $50 \times 5 \times 5$, FC1:500, FC2:10	99.1%
CIFAR10	CONV1: $32 \times 5 \times 5$, CONV2: $32 \times 5 \times 5$, CONV3: $64 \times 5 \times 5$, FC1:10	84.8%
CIFAR100	CONV1: $32 \times 3 \times 3$, CONV2: $64 \times 3 \times 3$, CONV3: $128 \times 3 \times 3$, FC1: 100	55.6%
ImageNet	AlexNet: [1]	54.2%
Clockwork [46]	LSTM [45]	88.9%

The Table only shows convolutional and fully connected layers as these layers take the bulk of computations in DNNs.

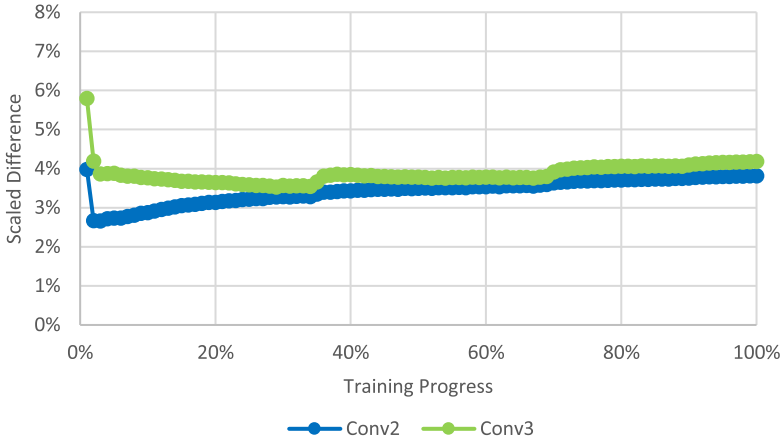


Fig. 3. Scaled difference in the second and third convolutional layers in CIFAR100.

We evaluate scaled difference in consecutive rows and columns of matrices involved in convolutional and fully connected layers. Scaled difference of two numbers in a matrix is defined as absolute value of Euclidean distance between the two numbers divided by the Euclidean distance of the maximum and minimum values of the matrix. If a matrix is the first operand of a multiplication, i.e., matrix A_i in Equation (1), then rows of the matrix are compared to calculate the scaled difference. However, if a matrix is the second operand of a matrix multiplication, i.e., F_i in Equation (1), then the columns of the matrix are compared to calculate scaled difference. Although comparing all rows or columns increases value similarity and results in more energy savings, it results in significant hardware overhead. Hence, we compare only consecutive rows or columns to keep the hardware overhead low.

On average, we found that scaled difference is 5.9% for the set of benchmarks in Table 2. Figure 3 shows the scaled difference for CIFAR100 for activations of the second and third convolutional layers. Consecutive rows and columns tend to be similar. In CIFAR100, the scaled distance ranges from 2.7% to 5.8%. Despite the small scaled difference, most of the 32-bit floating-point numbers are not exactly the same. This reduces value similarity. We define value similarity in matrix A as percentage of values in consecutive rows (if A is the first operand of a multiplication) or columns (if A is the second operand of a multiplication) of the A that are equal. One approach

for increasing values similarity is quantization. Quantizing elements of a matrix increase the degree of similarity, which in turn provides an opportunity to reduce redundant computations with negligible impact on accuracy.

In this work, we use linear quantization [4] for the matrices involved in the training of DNNs and increase value similarity. Linear quantization is a popular technique that maps a set of continuous values to a discrete set. To simplify implementation of quantization, we use uniformly distributed linear quantization for rows and columns of matrices in convolutional and fully-connected layers as these two layers are responsible for the majority of latency and delay. For matrix A, the quantization is applied according to the following equation to individual values

$$Quantized_{value} = \left\lfloor \frac{Value_{original}}{step} \right\rfloor \times step, \quad (5)$$

where the step is computed as a range of values in A divided by the number of clusters. The range of values is computed when a layer generates its outputs. For the first layer, the range is computed when inputs are read from memory. The quantized value is the cluster centroid that is closest to the original value. The benefit of quantization is that it increases value similarity of consecutive rows and columns with negligible impact on accuracy if the number of clusters is selected properly.

We exploit value similarity to increase the energy-efficiency of matrix multiplication. Assume that matrix A is multiplied by matrix B. For simplicity, we focus only on the first two rows of A. If the dot-product of the first row in A (r_0) and matrix B is saved, then the next row (r_1) that is the same as r_0 does not require any computation as row r_1 should be multiplied by the same matrix (B). In case, part of r_1 changes with respect to r_0 , the previous outputs can be corrected according to the following equation:

$$out_{new} = out_{old} + \sum_{i=0}^{n-1} ((a_{1i} - a_{0i}) \times B), \quad (6)$$

where a_{ij} represents an element in row i and column j of A and n is the number of columns in A. For each element in row r_1 , we calculate the difference between its quantized value and the corresponding quantized value in row r_0 ($a_{1i} - a_{0i}$). If the difference is 0, nothing needs to be done; otherwise, the difference should be multiplied by corresponding elements in matrix B and added to the saved output. If value similarity is high, then most of the values in r_0 and r_1 are the same. Hence, computing matrix multiplications based on Equation (6) significantly reduces number of MACs.

We define degree of computation reuse as (Total # of MACs)/(Remaining MACs) where the numerator is the total number of MACs in the baseline scheme and the remaining MACs is the number of MAC operations left after eliminating equal elements through quantization. Degree of computation reuse and value similarity are correlated since in case two consecutive rows have high value similarity, the results associated with the second row are mostly reused avoiding most of MAC operations.

One of the downsides of quantization is accuracy of DNNs. Although, clustering increases value similarity and provides more opportunities for computation reuse, it may increase error in DNNs predictions. We analyzed accuracy loss when the matrices involved in training of the DNNs are linearly quantized. We started with quantizing all three tensors (activation map, weight, and gradient) and observed that accuracy is degraded by a large margin. Then, we applied quantization to each of the three tensors, separately. Quantizing activation map has negligible impact on error if the number of clusters is selected properly. However, quantization of weight or gradient reduces

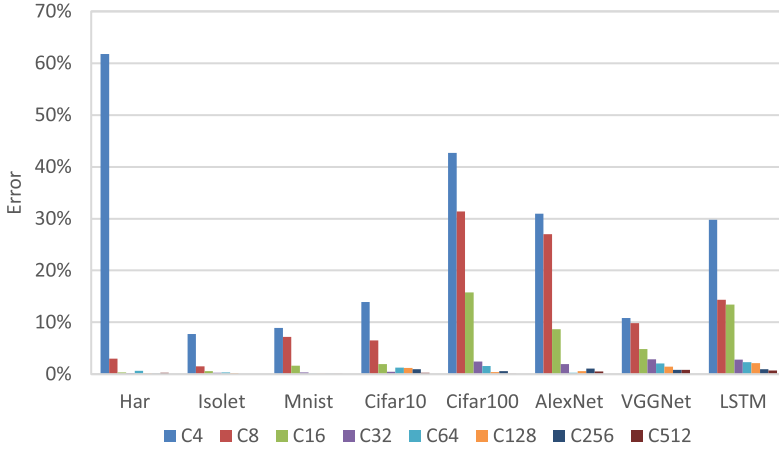


Fig. 4. Error vs. the number of clusters.

the accuracy of DNNs, significantly. Hence, for the rest of this article, we only focus on the quantization of activation map. However, we take into account latency and energy consumption of the other two tensors in our simulations.

One of the factors that impact accuracy of quantization is the number of clusters. Figure 4 shows error of DNNs compared to the baseline scheme ($\text{accuracy}_{\text{baseline}} - \text{accuracy}_{\text{quantized_scheme}}$) when the number of clusters changes from 4 to 512. When the number of clusters reduces, the value similarity increases as matrix elements are mapped to a smaller set of distinct values. However, it increases accuracy loss. Linear quantization with 4 clusters introduces significant accuracy loss in the majority of DNNs. The configuration with 16 clusters offers the best trade-off between similarity and error in HAR and ISOLET, MNIST, and CIFAR10 whereas in CIFAR100 and AlexNet, the configuration with 32 clusters offers the best results. In VGGNet and LSTM, the configuration with 64 clusters achieves a small accuracy loss.

From Figure 4, we conclude that there is no one-size-fit-all cluster size that works well for all DNNs. Even within a DNN, one cluster size may work better than the other one for a specific interval. In Section 4, we offer an adaptive technique that dynamically adjusts the number of clusters based on the error rate of layers during the training.

Figure 5 shows the degree of computation reuse for fully connected and convolutional layers. The number of clusters varies from 4 to 512. Computation reuse varies from 1.04 to 4.76. As the number of clusters increases, similarity in MACs reduces. This is mainly due to smaller cluster sizes which reduces the probability in which elements of consecutive rows or columns become equal. On average, the computation reuse based on the optimum cluster size per benchmark selected based on Figure 4 is 1.6. The results presented in Figure 5, are important as they indicate that exploiting computation reuse can potentially avoid a significant number of redundant computations. This provides an opportunity to accelerate DNNs and save energy by skipping redundant MACs.

4 COMPUTATION REUSED-BASED TRAINING FOR DNNs

This section describes the architecture of an accelerator which exploits a high degree of computation reuse characterized in Section 3 and improves performance and energy-efficiency in DNNs. First, we present an accelerator based on TC [17]. Then, we describe an extension to the TC needed to support computation reuse for DNNs.

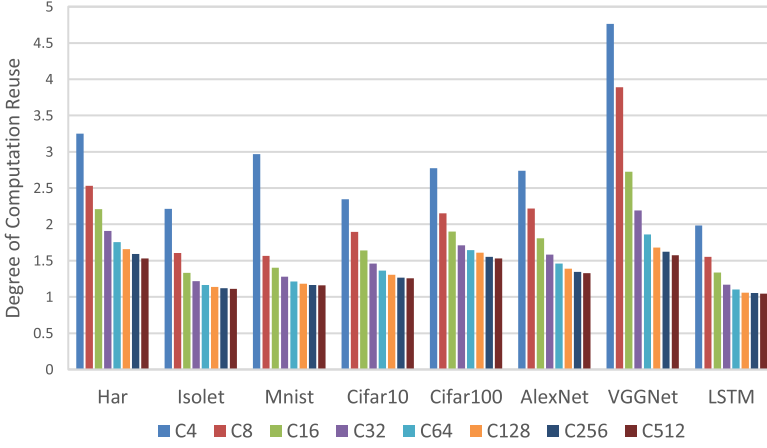


Fig. 5. Degree of computation reuse in DNNs.

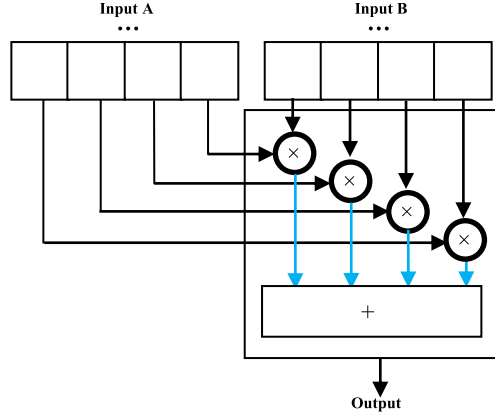


Fig. 6. A PE with 4 pairs of inputs and one output.

4.1 Tensor Core (TC) Accelerator

NVIDIA GPUs with TCs [17] are popular for training of DNNs. TCs are new additions to CUDA cores that facilitate implementation of MACs and offer significant computational horsepower for deep learning algorithms. Hence, in this work, we exploit an accelerator with an architecture similar to TCs, reflecting the functionality of the NVIDIA Volta V100 [17]. While the number of TCs changes from one GPU to the other, the building block of TCs more or less is the same. The bulk of a TC is a **processing element (PE)** that can perform 4 MACs per cycle, all contributing to the same output. Figure 6 shows structure of a PE. Inputs A and B can be a subset of matrices in Equations (1)–(3). As an example, they can be four pairs of (inputs, weights), all contributing to the same output or four pairs of (gradients, weights), all contributing to the same gradient. Such a structure for the PE is more energy-efficient than separate MACs as it reduces the cost of updating separate accumulators over several cycles by fusing the MACs. PEs operate on **16-bit floating-point (FP16)** inputs with **32-bit floating-point (FP32)** accumulations. Multiplication of FP16 numbers results in full-precision FP32 intermediate products that are then accumulated using 32-bit adders.

Each PE has three buffers, two for input operands and one for the output. Each buffer is composed of 2D SRAM cells. Each row of an input buffer provides operands for the multipliers. The

number of read ports is equal to the number of multipliers per PE, i.e., 4 in Figure 6. Each row of the buffer has a separate write port. When all numbers of a row are drained by the PE, then new values are read from on-chip caches and are written to the row. The number of rows per buffer is a design parameter and we will evaluate its impact on speedup in Section 6. Similar to a TC in NVIDIA Volta V100 [17], the baseline accelerator is composed of several tiles and each tile is implemented by a set of 4×4 PEs. Hence, a tile can compute a 4×4 matrix multiplication within a cycle (64 MACs per cycle).

4.2 Matrix Multiplication Using PEs

Each PE has a limited number of multipliers and is able to calculate dot products of a subset of rows and columns in matrices. We need to spread large matrices in DNNs across multiple PEs and exploit parallelism provided by TCs to accelerate training of DNNs. To calculate the multiplication of matrix A by matrix B, one approach would be computing dot product of a row from A and a column from B using a set of PEs. The row and the column are split into buffer sized chunks and then each PE generates a partial product. After computing partial products, they need to be added together. The addition of partial products requires communications among PEs which result in an extra delay in matrix multiplications. In addition, extra adders are needed to compute sum of partial results. The alternative approach would be allocating each PE for computation of dot product of an entire row from matrix A and an entire column from matrix B. Each time, buffer sized chunks from a row in A and a column in B are read from on-chip caches and are transferred to the input buffers of a PE (A and B in Figure 6). The PE computes the dot product. Then, the next chunks are read from on-chip memory and the outcome of the dot product is accumulated with the previous partial product. This procedure continues until all elements of the row in A and the column in B are processed. Since all partial products are accumulated by the same PE, no communication is needed among PEs for matrix multiplication. This method is faster than the first method and we use it for the rest of the article. Figure 7 depicts multiplication of two 16×16 matrices.

4.3 Computation Reuse in PEs

In this section, we present a hardware architecture that enables TCs to take advantage of the high degree of computation reuse found in DNNs. Our design exploits value similarity between consecutive rows or columns and saves computations and memory accesses. Figure 8(a) shows an example for multiplication of two matrices: A and B. For simplicity, we only focus on the first two rows of matrix A. In the second row, six out of eight elements are the same as the first row. Figure 8(b) shows how the two rows are multiplied by the first column in B (c0) when we do not attempt to exploit value similarity. We assume that each input buffer has two rows and each row has four entries, one for each multiplier in the PE. The figure shows that if the PE processes rows of A regardless of their values, then it is straightforward to arrange them in the buffer so that the PE can read them as groups of four numbers, performing four MACs per cycle. The PE needs two cycles to process each row.

For the second row of A, only two numbers are different from the first row. As far as the PE takes into account these two numbers in dot products, it can compute the correct result with less computations. To improve performance and reduce energy, the PE should skip MACs associated with equal values and compute results based on Equation (6). Figure 8(c) shows how to exploit computation reuse. The inputs of a layer are quantized when they are generated by the previous layer. We assume that the first layer is quantized when they are read from off-chip memory. By using the output of the PE left from the first row, MACs of the redundant elements for the second row are avoided (Equation (6)). This reduces energy of PEs as only two MACs are performed for the second row.

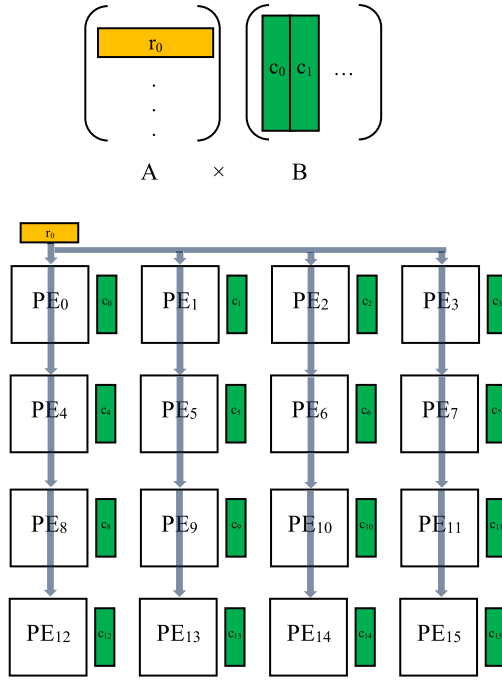
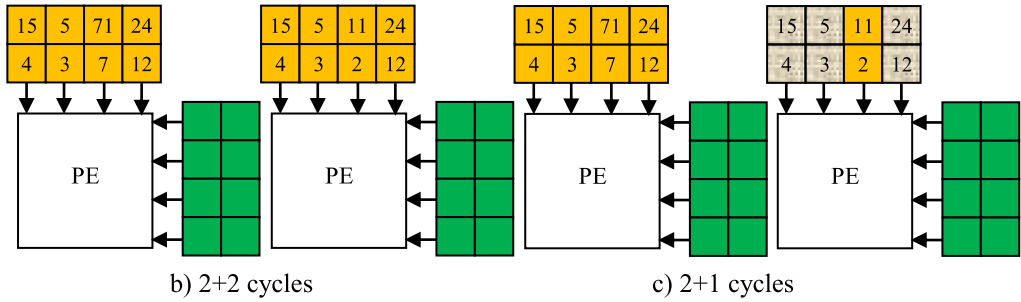


Fig. 7. Matrix multiplication for 16×16 matrices. We assume that each tile is composed of 4×4 PEs. Row 0 in A is multiplied by columns 0–15 from B. This is achieved through PE0–15 on a tile. This Figure shows how to compute the dot product of row 0 and matrix B. We need 15 more tiles to compute dot products of rows 1–15 of A and matrix B.

$$A = \begin{pmatrix} 4 & 3 & 7 & 12 & 15 & 5 & 71 & 24 \\ 4 & 3 & 2 & 12 & 15 & 5 & 11 & 24 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{pmatrix}$$

a)



b) 2+2 cycles

c) 2+1 cycles

Fig. 8. (a) Matrix A is multiplied by matrix B. Only A is shown in the Figure. We focus on dot products of the first two rows in A and the first column in B (c_0). (b) Multiplication of each row in A and c_0 requires 2 cycles. (c) With computation reuse, multiplication of the second row in A and c_0 requires only one cycle.

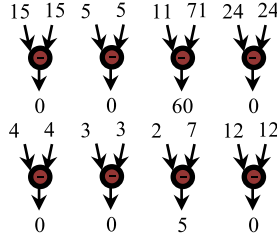


Fig. 9. Each subtractor has two inputs. The first input comes from the buffer and the second one comes from the shadow buffer. The output of a subtractor shows whether the corresponding values are the same in current and previous rows.

In addition to energy, this technique can improve the latency of matrix multiplication. To pack more MACs into the same cycle, a PE should shuffle the MACs so that the positions of redundant MACs are filled with non-redundant MACs. Ideally, the second row can be processed within a cycle (Figure 8(c)). However, this requires moving numbers in the two input buffers in a lock-step manner both in time (inter-row movements) and in space (intra-row movements). To take advantage of value similarity that appears in forward and backward passes, we need a solution that dynamically detects redundant MACs and eliminates them at runtime. The solution needs to be energy-efficient and should enable arbitrary moves of operands as well as rearranging tensors in different ways to suite different operations without using an exhaustive interconnect.

4.3.1 Computation Reuse-aware Tensor Core (CRTC). This section explains details of CRTC and how it skips MACs for redundant elements of matrix multiplications. While we assume that each PE in CRTC performs 4 MACs per cycle, the proposed hardware can be used to extend any data-parallel unit. Section 6 evaluates the sensitivity of CRTC to the width of PEs.

In CRTC, each buffer has a shadow buffer for elements of the previous row. Also, the output of the PE is saved into a buffer. The PE subtracts the first row (shadow buffer) from the second row (Figure 9). If the outcome of the subtraction is zero, there is no need for a MAC. In case an input in the two rows is different, the output of the PE should be updated. Since the PE stores the output of the previous row, correction is performed by removing the contribution of the previous row and adding the contribution of the new row. If there is enough degree of computation reuse, then the saving of this technique exceeds the overhead of extra components such as buffers and subtractors.

Each PE has a controller that receives a bit-vector from outputs of subtractors identifying non-zero values. For a 2-deep buffer, the bit-vector would be 8-bit wide. A multiplexer is connected to the input of each multiplier in a PE. The input of the multiplexer comes from the subtractors. Each cycle, the controller selects up to four subtractors where their outputs are non-zero. The controller generates appropriate values for select lines of the multiplexers. The same control signals are shared by the two input buffers of the PE. This is needed to assure that the PE processes operands of two input matrices in a lockstep manner.

Since CRTC skips redundant values, the input buffers should be able to feed multipliers with non-zero values continuously so that the maximum performance capacity of the underlying hardware is harnessed. For an input buffer with N rows, N write ports are needed, one per row so that an entire row can be filled if it is drained by multipliers. Hence, the on-chip caches are banked so that the high throughput needed for inputs of multipliers are sustained when multiple rows are drained out of the buffers at once. We will describe details of the hardware controller and PE configuration next.

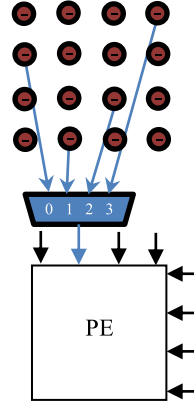


Fig. 10. The controller selects non-redundant elements of matrix rows or columns through the input multiplexers and sends them to PEs for dot product computations.

4.3.1.1 Controller for CRTC. Figure 10 shows the architecture of a PE in CRTC. The input buffer has four rows, each capable of holding four values. For each multiplier, there is a multiplexer with four inputs. The figure shows the multiplexer for the second multiplier. In addition to the second input which exists in the baseline scheme, there are three other inputs: one from the third column of the previous row, the first column of the second previous row, and the last column of the third previous row. The multiplexer of each multiplier has the same connectivity pattern which is shifted relative to its position. We assume that multiplexers are on the ring so the first and the last multiplexers are adjacent (connections wrap around the edges).

The controller receives a 16-bit vector indicating zeros generated by the subtractors. Then, the controller generates two sets of control lines for the two input buffers of the PE. Each set drives select lines of multiplexers which for Figure 10 is four 2-bit signals. As the values in the two input buffers should move in tandem, the select line of multiplexers are the same for the two buffers. The controller also generates an empty flag per row, indicating whether a row was drained so that it can be filled with data from on-chip caches.

The goal of the controller is to select inputs of the multiplexers so that PEs process their non-redundant values as quickly as possible. To do so, the controller follows a greedy approach and selects up to four non-zero inputs for multipliers per cycle. For each multiplexer, the controller uses a static priority: 1, 2, 0, and 3 (input of the multiplexer in Figure 10). The controller uses a 4–2 priority encoder for multiplexers. Each priority encoder receives four out of 16-bit zero vector and selects the first non-zero element based on the priority discussed above and produces 2-bit for the select line of the corresponding multiplexer.

The PE depicted in Figure 10 connects different buffer entries to the inputs of a multiplexer. In wider PEs with a greater number of multipliers, larger multiplexers are needed. It is possible that inputs of two different multiplexers come from the same position of the buffer. If the same number is selected by two multiplexers, then the contribution of a number is accounted twice in the output and this results in wrong matrix multiplication. To ensure that the controller generates correct control signals and avoids selecting overlapping inputs, we use four steps to generate the control signals. In each step, the select line of a multiplexer is determined. The buffer value that is selected in a step, its corresponding zero bit is set so that it is removed from the following steps. This simple technique prevents duplicated MACs for a buffer entry. While we described the procedure in steps, the controller is implemented using combinational components and operates within a single clock cycle.

During the training, the input activation matrix is used in two out of the three equations (Equations (1) and (3)). For example, the input activation is convoluted with the weight in the forward pass and is convoluted with the output gradient in the backward pass. In each equation, the group of activations that contribute to the output is different. This causes complications for memory layout. If the matrix appears in the equations in the same way, the matrix can be stored in the memory hierarchy, statically. However, during the forward and backward passes, the order in which elements of a matrix are accessed changes. As a result, a memory layout that fits perfectly for a forward pass may not work well for the backward pass. To overcome this challenge, we exploit a transposer unit to swap rows and columns of matrices whenever it is needed. For example, for the PE depicted in Figure 10, we use a transposer that accommodates 4×4 values. When elements of a matrix are fetched from off-chip memory, they are transferred into banked on-chip caches. Then, the transposer reads from on-chip caches, converts the layout to column-wise, and writes them into buffers.

We assume that the granularity of accesses to on-chip caches is a group of four numbers which is the same as number of elements per row in a PE. As a result, a transposer can access any groups of 4-value in the banks. A transposer reads four 4-value blocks from their corresponding banks and copies them into its internal 4×4 storage. The transposer then can provide four blocks of 4-value where each value within a block is taken from a different input block, effectively transposing the elements.

4.4 Adaptive CRTC (ACRTC)

We exploit quantization to increase similarity in rows and columns of tensors involved in matrix multiplications. One factor that impacts both accuracy and speedup of DNNs is the number of clusters. Using a large number of clusters, while desirable from accuracy point of view, restricts speedup and energy savings. Based on data presented in Figure 4, a static approach for cluster size requires profiling as the best cluster size varies from one DNN to the other. In this section, we offer an adaptive technique which adjusts the number of clusters dynamically and during the runtime.

We monitor the error of a DNN during the training phase in conjunction with a saturating counter to adjust the number of clusters. The value of an n -bit saturating counter changes from 0 to $2^n - 1$. After forward and backward passes for a batch of inputs, we compare the error of the network with the error of the previous batch. If it increases then the saturating counter is reset to zero; otherwise, it is incremented. If the error of the network increases for the current batch of inputs, then the number of clusters is multiplied by two only if the saturating counter is less than a predetermined threshold. We found that using a saturating counter prevents adjusting cluster size for transient changes in error of DNNs. Similarly, we divide the cluster size by two if the error of the DNN reduces and the saturating counter is more than the threshold. From Figure 4, increasing the number of clusters beyond 512 does not improve accuracy. Hence, we restrict the cluster size to be less than or equal to 512.

5 EVALUATION METHODOLOGY

Our objective is to prove that ACRTC provides savings on the state-of-the-art DNNs covering a wide range of applications. To this end, we evaluate ACRTC with popular NN applications: activity recognition trained on HAR dataset [26], voice recognition trained on ISOLET dataset [25], handwriting classification trained on MNIST dataset [24], object recognition trained on CIFAR datasets [27], image classification trained on ImageNet dataset [29], and LSTM [45] as a recurrent NN trained based on clockwork RNN [46]. Table 2 presents the set of DNNs used in this work as well as their topologies. For well-known applications such as CIFAR, we used the model suggested by the Caffe Model Zoo library [31]. We also used the same library for architecture of AlexNet [1]

Table 3. Configuration of ACRTC Accelerator

Technology	45 nm
Frequency	500 Mhz
# of TCs	256
# of PEs per TC	16
# of MACs/cycle within a PE	4
Off-chip memory	16 GB LPDDR4
On-chip cache	3 caches, each 128 KB $\times$ 64 banks
SRAM buffers	4 rows $\times$ 4 columns

and VGGNet [30]. For other network architectures, we selected topologies that result in an acceptable level of accuracy. The accuracy of a network is defined as the number of correctly classified data to the total number of elements in a testing dataset. The last column in Table 2 shows accuracy of DNNs for the baseline scheme. Each DNN is trained using stochastic gradient descent with momentum [32]. The activation function in all hidden layers is set to **Rectified Linear Unit (ReLU)** and for output layers is set to SoftMax. We used the same number of epochs in the baseline and ACRTC accelerator for training of DNNs. This is necessary to assure a fair comparison between the two accelerators.

We developed a cycle-accurate simulator to model the hardware presented in Section 4. We verify the simulator by running several DNNs and validating the timing fidelity of the simulator. Table 3 shows parameters of the simulator. To estimate energy consumption, we model the accelerator in Verilog and synthesize it using the 45 nm FreePDK library [21]. ACRTC increases the area of the chip from 66.3 mm² in the baseline to 70.4 mm². We use CACTI [22] to estimate energy of on-chip SRAM memories which are heavily banked to increase memory-level parallelism. We also use CACTI [22] to model energy of SRAM buffers. For all SRAMs, we assume the itrs-lop model as this decreases energy per access, while offering an acceptable level of memory latency. Finally, the energy consumption of the main memory is estimated by using the MICRON power model for LPDDR4 [23]. The power of hardware components estimated by the aforementioned tools is integrated with activity factors provided by our simulator to obtain the total energy of the accelerator.

6 EVALUATION RESULTS

This section reports simulation results for our proposed techniques. First, we present speedup and energy saving under ACRTC. Next, we evaluate the sensitivity of ACRTC to architectural parameter such as size of saturating counters and width of PEs.

6.1 Speedup

Figure 11 reports normalized speedup achieved by ACRTC in TC architecture. Bars more than one indicate speedup under ACRTC. For each DNN, the number of bits for saturating counters varies from 1 to 4. For a saturating counter with n -bit, we set a threshold equal to $2^{(n-1)} - 1$. ACRTC provides consistent speedup for all DNNs, ranging from 1.06 $\times$ to 2.04 $\times$. Execution time reduction is the result of redundancy in MACs. If two consecutive rows or columns have equal quantized values, then corresponding multiplications are avoided. In addition, the overhead of quantization and subtraction of consecutive rows or columns is small as it is performed per input element not per connection. As an example, in a fully-connected layer, each input is connected to hundreds to thousands of neurons and an equality check for an input can potentially avoid hundreds to thousands of costly multiplications.

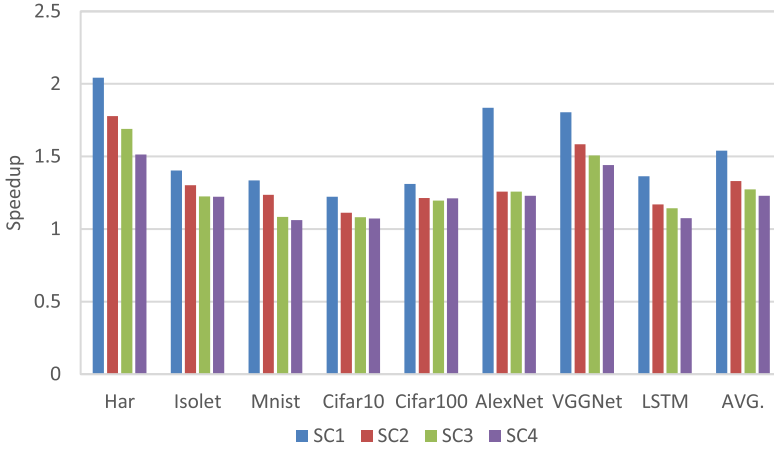


Fig. 11. Speedup of ACRTC over the baseline scheme. The number of bits in saturating counters changes from 1 to 4.

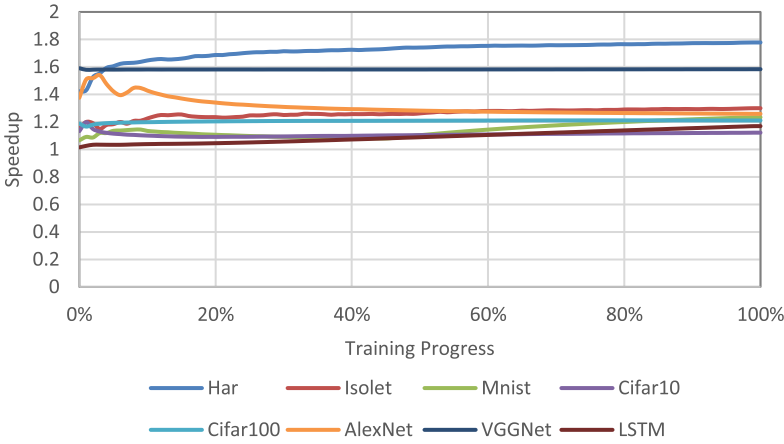


Fig. 12. Speedup with ACRTC as training progresses.

As the number of bits in saturating counters increases, speedup reduces. This is expected as a larger saturating counter requires more iterations to pass its corresponding threshold. As a result, it is less likely that the number of clusters reduces in a larger saturating counter compared to a smaller one. This decreases probability of having two quantized elements become equal which in turn reduces speedup.

Figure 12 reports speedup of ACRTC during the training from the first epochs all the way to the last one. Overall, speedup is stable throughout the entire training which indicates that the underlying phenomena for redundant MACs is not transient and is independent of the choice of network topology. In some DNNs such as AlexNet, speedup fluctuates at the start of training. This behavior is related to the random initialization of the model. However, as training progresses, speedup stabilizes as the model quickly learns which features of input data are important and masks the rest. This translates to stable value similarity as the set of features that the network focuses on narrows down.

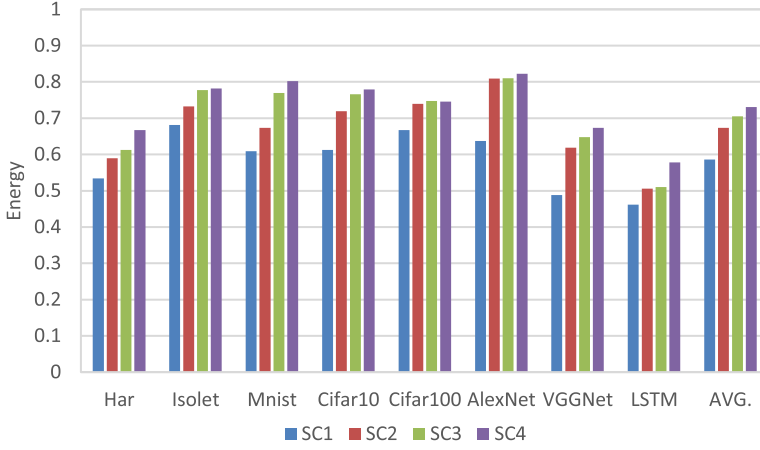


Fig. 13. Energy saving of ACRTC over the baseline scheme.

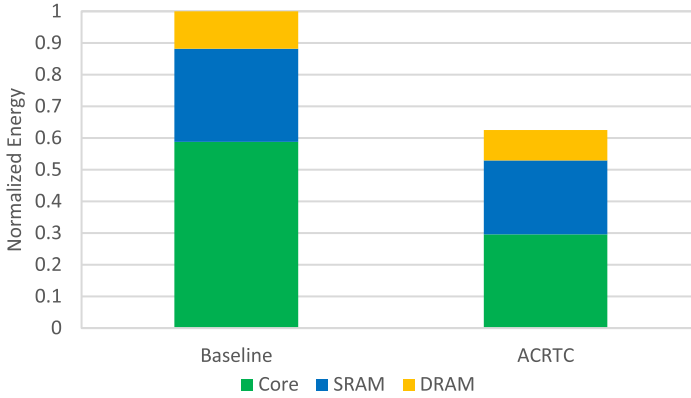


Fig. 14. Energy consumption breakdown in the baseline accelerator and ACRTC.

6.2 Energy Saving

Figure 13 reports the energy consumed by ACRTC relative to the baseline scheme. On average, energy reduces from 27% to 41% when the number of saturating counter bits changes from 4 down to 1. There are two major sources for energy saving. The first one is related to the activity reduction in both computing cores and memory units. Redundant MACs do not need multiplications which reduce dynamic power. Also, quantization of input tensors reduces dynamic energy of SRAM memories. The other source of energy reduction is associated with performance improvement reported in Figure 11. ACRTC accelerates training of DNNs which in turn reduces static energy.

Figure 14 shows breakdown of energy consumed across the three main components: core, on-chip SRAM memories, and off-chip DRAM memory. The Figure reports energy breakdown for both the baseline scheme and the ACRTC. The overhead of ACRTC such as multiplexers, subtractors, and shadow buffers is included in the energy of the core and SRAM. The Figure shows the aggregate results for all DNNs. Most of the energy saving is associate with the core and on-chip memories. ACRTC reduces energy of core and on-chip memories by 29% and 6%, respectively. Most of the energy saving in the main memory is related to static power as tensors are saved into

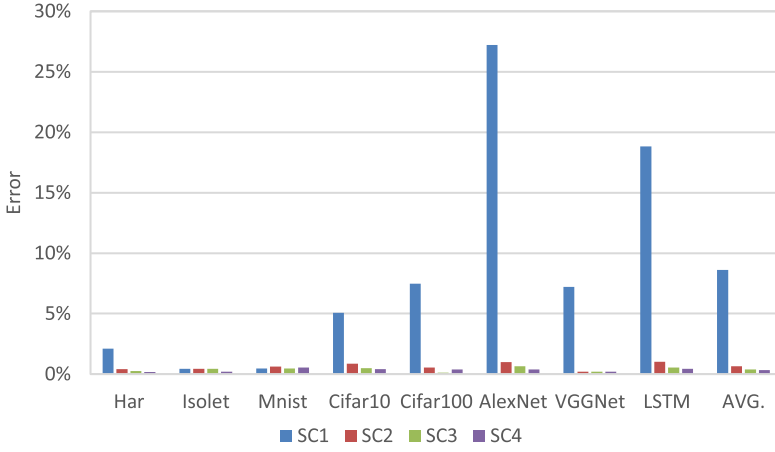


Fig. 15. Sensitivity of error to the size of saturation counters.

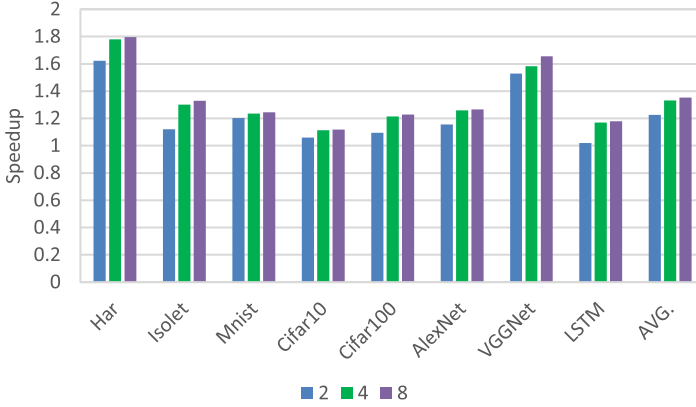


Fig. 16. Sensitivity of ACRTC to multiplexers with variable number of inputs.

the main memory with full-precision. Only on-chip memories are used to accommodate quantized tensors.

6.3 Accuracy

In this section, we analyze the sensitivity of ACRTC to the number of bits in saturating counters. Figure 15 shows error of ACRTC compared to the baseline scheme (error = accuracy_{baseline} – accuracy_{ACRTC}) when the size of counters changes from 1 to 4. For an n -bit counter, we set the threshold to $2^{(n-1)} - 1$. As an example, in a 4-bit saturating counter, threshold is equal to 7. We tried different values for saturating counters and found that threshold of $2^{(n-1)} - 1$ results in the best trade-off between accuracy and speedup/energy saving. In addition, such a configuration for the saturating counter reduces overhead of implementation as comparison of the threshold and the counter requires only checking the **most-significant bit (MSB)**. If MSB is one, then the saturating counter exceeds the threshold. Accuracy improves as the number of bits in the saturating counters increases. This is expected as it is less likely that the value of a counter passes its threshold when the size of the counter increases. In other words, ACRTC becomes more conservative and shrinks cluster size less often in larger saturating counters. However, it is not desirable to make a counter

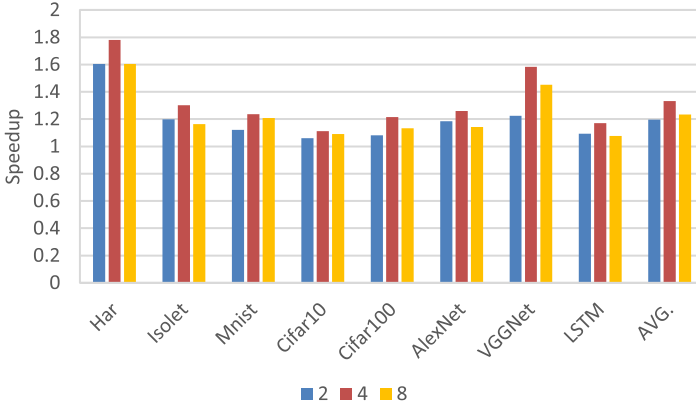


Fig. 17. Sensitivity of ACRTC to the width of PEs.

too large as it has a detrimental impact on speedup as well as energy saving. On average, error is 8.5%, 0.6%, 0.4%, and 0.3% when the number of bits changes from 1 to 4. To maximize speedup and energy saving, we select the smallest saturating counter with accuracy similar to the baseline scheme which is a 2-bit saturating counter.

6.4 Sensitivity to Multiplexers

In the event of similarity in tensors, multiplexers shuffle inputs of the multipliers to feed multipliers with non-zero operands. The choice of multiplexer size impacts speedup. A small multiplexer with a limited number of inputs limits speedup as some non-zero elements may exist in input buffers, while they cannot be fed to the multipliers due to unavailability of connections. In this section, we evaluate the sensitivity of ACRTC to 2-, 4-, and 8-input multiplexers. For each multiplexer size, we tried several configurations for connectivity patterns and selected the configuration with the highest speedup. ACRTC improves performance by 1.23 $\times$, 1.33 $\times$, and 1.35 $\times$ over the baseline scheme for 2-, 4-, and 8-input multiplexers, respectively. A 2-input multiplexer misses many opportunities to utilized multipliers and keeps them busy with non-zero operands. However, a 4-input multiplexer boosts speedup by 10%. Since each input buffer accommodates 16 operands, four 4-input multiplexers can cover the entire buffer if the inputs of the multiplexers are spread across the buffer entries (Figure 10). The increasing size of multiplexers from 4 to 8 has a marginal impact on speedup. This is expected as four 8-input multiplexers cover buffer entries redundantly without providing extra connectivity compared to 4-input multiplexers.

6.5 Sensitivity to the Width of PEs

We study the impact of wider PEs that have more multipliers on speedup. Figure 17 shows normalized speedup in ACRTC when the size of PEs changes from 2 to 8. To have a fair comparison, we reduce the number of TCs in larger PEs. For example, there are 256 TCs for PE width of 4 (Table 3). When the size of PE doubles, we cut the number of TCs by half so that the total number of multipliers stays the same. Also, the size of multiplexers increases with PE width, i.e., TCs with PE width of two use 2-input multiplexers, TCs with PE width of 4 use 4-input multiplexers, and so on. When PE width is two, there are limited opportunities to exploit value similarity as there are only two rows, each with two columns within a buffer. However, TCs with PE width of 4 are able to exploit value similarity and achieve higher speedup than PE width of 2. For PE width of 8, it is harder to drain rows of buffers as the probability of having equal rows within a buffer reduces. This explains the performance drop from PE width of 4–8.

7 RELATED WORK

DNNs became popular in a wide range of applications and different computation devices run machine learning algorithms such as mobile devices [33] and data centers [34]. DNNs are power hungry applications and require significant horsepower from underlying hardware. As a result, numerous accelerators are proposed to improve energy-efficiency of DNNs. DianNao [41] and its variants DaDianNao [15] for large scale applications and ShiDianNao [33] for mobile devices are the earliest accelerators which exploit on-chip memory to reduce the impact of memory latency on performance. Scaleddeep [11] exploits a scalable architecture with heterogeneous tiles and low overhead synchronization to accelerate DNNs. Recent accelerators focus on power consumption. Common techniques to reduce power include reducing the precision of functional units and data [37], pruning connections or neurons [36], and reducing voltage in exchange for reliability [16]. UCNN [38] accelerates inference of CNNs by exploiting weight repetition. Reducing computation via weight repetition is inspired by the way CNN filters are designed. If the size of a filter exceeds the number of unique weights, then it is guaranteed that repeated weights exist. Factoring repeated weights reduces computations in CNNs. Riera et al. [35] accelerate sequence processing problems. Sequence problems refer to the applications where inputs overlap over time. An example would be a video file where two consecutive frames share a large number of equal or similar pixels. For these applications, computations of similar data are avoided during the inference. These techniques improve power by reducing computations in neurons only during the inference. However, energy saving in the training phase is a challenging task due to dynamic nature of network parameters. ACRTC is different as it exploits redundancy that exists between rows and columns of tensors to save energy and improve speed during the training of DNNs.

Training of a DNN involves a significant amount of computations over a large number of training iterations so that the network converges. While there are a large number of research papers on inference, only a handful of prior works focused on the performance and/or energy of training. Caterpillar [39] proposes an architecture with hierarchical support for collective communication semantics, which provides the flexibility needed to efficiently train networks with both stochastic and batched gradient descent-based techniques. NXT [40] exploits a near-memory architecture in conjunction with heterogeneous cores to accelerate training. The data-path of NXT is built around full-precision fused accumulators and supports a rich set of arithmetic operations needed for both forward and backward passes such as matrix operations and scatter operations. **Locality Sensitive Hashing (LSH)** [19] is used to reduce computations in training phase. For a given matrix, several LSH functions map an input into a binary vector. Then, the binary vector is used to reduce computation in matrix multiplications. This technique is a software-based approach and implementation of this technique in hardware results in significant overhead as it requires a large number of hashing and comparators. Exploiting sparsity in DNNs is another approach for acceleration of DNNs. Skipping computation and storage associated with sparsity in weights [2, 9], activations [8, 18], or both [18, 36] are used to save latency and energy. Cerebras [14] exploits thousands of **Sparse Linear Algebra (SLA)** cores with capability of filtering out zeros from computations. Cerebras [14] cores are equipped with schedulers, operating at the granularity of a single operand so that only non-zero data triggers computations. As a result, SLA cores never multiply a number by 0 and never propagate zeros across computational units. Randomized hashing [10] relies on hash tables to reduce multiplications. This technique maps neuron weights into hash tables and selects neurons with highest activations. Hence, instead of processing all activations, only a subset of them is selected, leading to computation savings. We use a different approach for latency and energy enhancement in DNNs. We exploit value similarity exists in activation tensors to reduce redundancy in neuron computations.

There have been several implementations of DNN training on FPGAs. Girones et al. [42] propose a pipelined architecture for the backpropagation algorithm. The proposed hardware is evaluated only for a perceptron-based neural network. F-CNN [43] is a reconfigurable framework for training of CNNs. F-CNN reconfigures a data-path at runtime to cover different layers in forward and backward passes. One of the shortcomings of the F-CNN is that it is not scalable and cannot be used for a cluster of FPGAs. FPDeep [44] uses a cluster of FPGAs for training of CNNs. Each FPGA implements a set of layers or a subset of a layer depending on how large a layer is. The cluster of FPGAs acts as a pipeline and reduces the number of accesses to the off-chip memories. While in this work, we target custom-built accelerators, our proposed techniques can be deployed into FPGAs.

Some training algorithms reduce the number of iterations required to converge. You et al. [3] increase batch size to accelerate training of DNNs based on ImageNet dataset. Batch normalization [20] accelerates training of DNNs by reducing the impact of input changes on training time. As the distribution of the primary inputs changes, layers need to continuously adapt to the new distributions, which slow down training. Batch normalization reduces the dependence of gradients on the scale of the network parameters. This allows using higher learning rates, reducing training time without risk of divergence. Importance sampling [28] is another technique to reduce convergence time in DNNs. Not all elements of a training dataset are equally important. Some of them require a few epochs for training and some can be ignored without impacting the accuracy of the final model. Katharopoulos et al. [28] propose using loss rate to sample a subset of input for training. ACRTC is orthogonal to the aforementioned techniques and can be merged with them to accelerate training of DNNs, further.

8 CONCLUSION

This article presents ACRTC as a technique to reduce the computation cost of training in DNNs. Experiments conducted on a wide range of DNNs show that rows and columns of inputs exhibit a high degree of similarity. We propose a new scheme that exploits this similarity and avoids a large number of MACs by reusing results of previous executions. We envaulted inputs of DNN layers and observed that using quantization, on average, 70% of inputs in fully-connected and convolutional layers remain unmodified. Based on this observation, we propose a low-level PE for accelerators.

The proposed scheme spreads matrix multiplications across PEs and checks consecutive rows and columns of input tensors. Those elements that remain unmodified are ignored, avoiding associated computations and saving latency and energy. Modified inputs are used to correct previous results. We also introduce an adaptive technique which changes the number of clusters dynamically using saturating counters. This avoids lengthy profiling for tuning the number of clusters per network. We implement ACRTC on top of a state-of-the-art accelerator architecture: TC. Our implementation requires trivial components such as multiplexers and subtractors. Experiments show that ACRTC accelerates training by 33% and saves energy by 32% with negligible impact on accuracy of DNNs.

REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton. 2012. Imagenet classification with deep convolutional neural networks. In *Proceedings of the Advances in Neural Information Processing Systems*. 1097–1105.
- [2] S. Zhang, Z. Du, L. Zhang, H. Lan, S. Liu, L. Li, Q. Guo, T. Chen, and Y. Chen. 2016. Cambricon-x: An accelerator for sparse neural networks. In *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture*.
- [3] Y. You, Z. Zhang, C.-J. Hsieh, J. Demmel, and K. Keutzer. 2018. Imagenet training in minutes. In *Proceedings of the 47th International Conference on Parallel Processing* 2018.
- [4] B. Widrow, I. Kollar, and M. C. Liu. 1996. Statistical theory of quantization. *IEEE Transactions on Instrumentation and Measurement* 45, 2 (1996), 353–361.

- [5] A. Jain, A. Phanishayee, J. Mars, L. Tang, and G. Pekhimenko. 2018. Gist: Efficient data encoding for deep neural network training. In *Proceedings of the 45th Annual International Symposium on Computer Architecture*. Piscataway, NJ, IEEE Press, 776–789.
- [6] R. D. Evans, L. F. Liu, and T. Aamodt. 2020. JPEG-ACT: A frequency domain lossy DMA engine for training convolutional neural networks. In *Proceedings of the 47th Annual International Symposium on Computer Architecture*. ACM, 860–873.
- [7] J. Dean, G. S. Corrado, R. Monga, K. Chen, M. Devin, Q. V. Le, M. Z. Mao, M. Ranzato, A. Senior, P. Tucker, K. Yang, and A. Y. Ng. 2012. Large scale distributed deep networks. In *Proceedings of the 25th International Conference on Neural Information Processing Systems - 1*. Curran Associates Inc., 1223–1231.
- [8] J. Albericio, P. Judd, T. Hetherington, T. Aamodt, N. E. Jerger, and A. Moshovos. 2016. Cnvlutin: Ineffectual-neuron-free deep neural network computing. In *2016 Proceedings of the ACM/IEEE 43rd Annual International Symposium on Computer Architecture*.
- [9] J. Zhang, X. Chen, M. Song, and T. Li. 2019. Eager pruning: Algorithm and architecture support for fast training of deep neural networks. In *Proceedings of the 46th International Symposium on Computer Architecture*. New York, NY. ACM, 292–303.
- [10] R. Spring and A. Shrivastava. 2017. Scalable and sustainable deep learning via randomized hashing. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 445–454.
- [11] S. Venkataramani, A. Ranjan, S. Banerjee, D. Das, S. Avancha, A. Jagannathan, A. Durg, D. Nagaraj, B. Kaul, P. Dubey, and A. Raghunathan. 2017. ScaleDeep: A scalable computer architecture for learning and evaluating deep networks. In *Proceedings of the 44th Annual International Symposium on Computer Architecture, Ser. ISCA'17*. New York, NY. ACM, 13–26.
- [12] E. Strubell, A. Ganesh, and A. McCallum. 2019. Energy and policy considerations for deep learning in NLP. Retrieved from CoRR abs/1906.02243, 2019.
- [13] Norman P. Jouppi et al. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture, Ser. ISCA'17*. New York, NY. ACM, 1–12.
- [14] “Cerebras CS1” 2019. Retrieved from <https://www.cerebras.net/product/>.
- [15] Yunji Chen, Tao Luo, Shaoli Liu, Shijin Zhang, Liqiang He, Jia Wang, Ling Li, Tianshi Chen, Zhiwei Xu, Ninghui Sun, and Olivier Temam. 2014. Dadiannao: A machine-learning supercomputer. In *Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture*.
- [16] B. Reagen, P. Whatmough, R. Adolf, S. Rama, H. Lee, S. K. Lee, J. M. Hern´andez-Lobato, G. Y. Wei, and D. Brooks. 2016. Minerva: Enabling lowpower, highly-accurate deep neural network accelerators. In *Proceedings of the 43rd Annual ACM/IEEE International Symposium on Computer Architecture*.
- [17] “NVIDIA Tesla V100 GPU Architecture,” 2017. Retrieved August 2017 from <https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf>.
- [18] Mostafa Mahmoud, Isak Edo, Ali Hadi Zadeh, Omar Mohamed Awad, Gennady Pekhimenko, Jorge Albericio, and Andreas Moshovos. 2020. Tensordash: Exploiting sparsity to accelerate deep neural network training. In *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture*. 781–795.
- [19] L. Ning, H. Guan, and X. Shen. 2019. Adaptive deep reuse: Accelerating CNN training on the fly. In *Proceedings of the 35th IEEE International Conference on Data Engineering* 1538–1549.
- [20] Sergey Ioffe and Christian Szegedy. 2015. Batch normalization: accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning*. 448–456.
- [21] FreePDKTM process design kit. Retrieved from <http://www.eda.ncsu.edu/wiki/FreePDK>.
- [22] N. Muralimanohar, Rajeev Balasubramanian, and Norm Jouppi. 2007. Optimizing NUCA organizations and wiring alternatives for large caches with CACTI 6.0. In *Proceedings of the 40th Annual IEEE/ACM International Symposium on Micro-architecture*. 3–14.
- [23] Micron lpdrr4 system power calculator. Retrieved from https://media-www.micron.com/-/media/client/global/documents/products/power-calculator/DDR4_power_calc.xlsm?la=en&rev=5e97be39078d4a1b8619cb85c96bbe63.
- [24] Y. LeCun, C. Cortes, and C. J. Burges. 1998. The mnist database of handwritten digits. Retrieved from <http://yann.lecun.com/exdb/mnist/>.
- [25] UCI machine learning repository. Retrieved from <http://archive.ics.uci.edu/ml/datasets/ISOLET>.
- [26] UCI machine learning repository Retrieved from <https://archive.ics.uci.edu/ml/datasets/Daily+and+Sports+Activities>.
- [27] The CIFAR dataset. Retrieved from <https://www.cs.toronto.edu/~kriz/cifar.html>.
- [28] Angelos Katharopoulos and François Fleuret. 2018. Not all samples are created equal: Deep learning with importance sampling. In *Proceedings of the 35th International Conference on Machine Learning*. 2530–2539.
- [29] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei. 2014. ImageNet large scale visual recognition challenge. Retrieved September, 2014 from CoRR abs/1409.0575

- [30] Karen Simonyan and Andrew Zisserman. 2015. Very deep convolutional networks for large-scale image recognition. In *Proceedings of the 32nd International Conference on Machine Learning*. 1–14.
- [31] Caffe model zoo. Retrieved from <https://github.com/BVLC/caffe/wiki/Model-Zoo>, 2020.
- [32] I. Sutskever, J. Martens, G. E. Dahl, and G. E. Hinton. 2013. On the importance of initialization and momentum in deep learning. In *Proceedings of the 30th International Conference on Machine Learning*. 1139–1147.
- [33] Z. Du, R. Fasthuber, T. Chen, P. Ienne, L. Li, T. Luo, X. Feng, Y. Chen, and O. Temam. 2015. Shidiannao: Shifting vision processing closer to the sensor. In *Proceedings of the ACM/IEEE 42nd Annual International Symposium on Computer Architecture*.
- [34] W. Xiong, J. Droppo, X. Huang, F. Seide, M. Seltzer, A. Stolcke, D. Yu, and G. Zweig. 2016. Achieving human parity in conversational speech recognition. Retrieved from CoRR 2016.
- [35] M. Riera, J.-M. Arnau, and A. Gonzalez. 2018. Computation reuse in DNNs by exploiting input similarity. In *Proceedings of the 45th IEEE Annual International Symposium on Computer Architecture*.
- [36] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally. EIE: Efficient inference engine on compressed deep neuralnetwork. In *Proceedings of the 43rd International Symposium on Computer Architecture*.
- [37] P. Judd, J. Albericio, and A. Moshovos. 2017. Stripes: Bit-serial deep neural network computing. In *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture*.
- [38] K. Hegde, J. Yu, R. Agrawal, M. Yan, M. Pellauer, and C. Fletcher. 2018. Ucn: Exploiting computational reuse in deep neural networks via weight repetition. In *Proceedings of the ACM/IEEE 45th Annual International Symposium on Computer Architecture*. 674–687.
- [39] Y. Li and A. Pedram. 2017. CATERPILLAR: Coarse grain reconfigurable architecture for accelerating the training of deep neural networks. In *Proceedings of the 28th IEEE International Conference on Application-specific Systems, Architectures and Processors*.
- [40] F. Schuiki, M. Schaffner, F. K. Gürkaynak, and L. Benini. 2019. A scalable near-memory architecture for training deep neural networks on large in-memory datasets. *IEEE Transactions on Computers* 68, 4 (2019), 484–497.
- [41] T. Chen, Z. Du, N. Sun, J. Wang, C. Wu, Y. Chen, and O. Temam. 2014. DianNao: A small-footprint high-throughput accelerator for ubiquitous machine-learning. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems*.
- [42] R. G. Girones, Rafael Gadea Gironés, Ricardo Colom Palero, Joaquín Cerdá Boluda, Joaquín Cerdá Boluda, and Angel Sebastia Cortés. 2005. FPGA implementation of a pipelined on-line backpropagation. *Journal of VLSI Signal Processing Systems for Signal, Image and Video Technology* 40, 2 (2005), 189–213.
- [43] Wenlai Zhao, Haohuan Fu, Wayne Luk, Teng Yu, Shaojun Wang, Bo Feng, Yuchun Ma, and Guangwen Yang. 2016. F-cnn: An FPGA-based framework for training convolutional neural networks. In *Proceedings of the 27th IEEE International Conference on Application-specific Systems, Architectures and Processors*. 107–114.
- [44] T. Geng, T. Wang, A. Li, X. Jin, and M. Herbordt. 2020. FPDeep: Scalable acceleration of CNN training on deeply-pipelined FPGA clusters. *IEEE Transactions on Computers* 69, 8 (2020), 1143–1158.
- [45] S. Hochreiter and J. Schmidhuber. 1997. Long short-term memory. *Neural Computation* 9, 8 (1997), 1735–1780.
- [46] Jan Koutnik, Klaus Greff, Faustino Gomez, and Juergen Schmidhuber. 2014. A clockwork RNN. In *Proceedings of the International Conference on Machine Learning*. 1863–1871.

Received June 2021; revised August 2021; accepted August 2021